Creating, Reading, Updating, and Deleting (CRUD)

Names such as SQLite and MySQL emphasize that all access to a database is via the Structured Query Language (SQL). In most cases, Rails will take care of this for you, but that's completely up to you. As you'll see, you can provide clauses or even entire SQL statements for the database to execute.

If you're familiar with SQL already, as you read this section take note of how Rails provides places for familiar clauses such as `select`, `from`, `where`, `group by`, and so on. If you're not already familiar with SQL, one of the strengths of Rails is that you can defer knowing more about such things until you actually need to access the database at this level.

In this section, we'll continue to work with the `Order` model from the Depot application for an example. We'll be using Active Record methods to apply the four basic database operations: create, read, update, and delete.

Creating New Rows

Given that Rails represents tables as classes and rows as objects, it follows that we create rows in a table by creating new objects of the appropriate class. We can create new objects representing rows in our `orders` table by calling `Order.new`. We can then fill in the values of the attributes (corresponding to columns in the database). Finally, we call the object's `save` method to store the order back into the database. Without this call, the order would exist only in our local memory.

rails7/e1/ar/new_examples.rb

```
an_order = Order.new
an_order.name     = "Dave Thomas"
an_order.email    = "dave@example.com"
an_order.address  = "123 Main St"
an_order.pay_type = "check"
an_order.save
```

Active Record constructors take an optional block. If present, the block is invoked with the newly created order as a parameter. This might be useful if you